

Mathematical Interpretation of Vision-Based UAV Target Tracking

1 Overview

This project implements a vision-based moving-target tracking system for a quadrotor UAV in a PyBullet simulation. A colored target moves through the environment and is observed using the UAV's camera. Computer vision detects the target in the image, and a feedback controller uses the detected image position and apparent target size to control the UAV.

The complete mathematical operation is

$$\boxed{\text{Target Motion} \rightarrow \text{Camera Observation} \rightarrow \text{Target Detection} \rightarrow \text{Tracking Error} \rightarrow \text{PD Control} \rightarrow \text{UAV}} \quad (1)$$

Thus, the project is a closed-loop visual feedback control system.

2 1. Moving Target Model

The target position in world coordinates is

$$\mathbf{p}_T(t) = \begin{bmatrix} x_T(t) \\ y_T(t) \\ z_T(t) \end{bmatrix}. \quad (2)$$

For the default circular trajectory used in the code,

$$x_T(t) = x_C + R \cos(\omega t), \quad (3)$$

$$y_T(t) = y_C + R \sin(\omega t), \quad (4)$$

$$z_T(t) = h. \quad (5)$$

Therefore,

$$\boxed{\mathbf{p}_T(t) = \begin{bmatrix} x_C + R \cos(\omega t) \\ y_C + R \sin(\omega t) \\ h \end{bmatrix}} \quad (6)$$

where R is the trajectory radius, ω is the speed parameter, (x_C, y_C) is the trajectory centre, and h is the target altitude.

For the current implementation,

$$R = 1.5 \text{ m}, \quad \omega = 0.4, \quad h = 1.0 \text{ m}. \quad (7)$$

The target velocity is

$$\mathbf{v}_T(t) = \frac{d\mathbf{p}_T(t)}{dt} = \begin{bmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \\ 0 \end{bmatrix}. \quad (8)$$

3 2. UAV–Target Geometry

Let the UAV position be

$$\mathbf{p}_D(t) = \begin{bmatrix} x_D(t) \\ y_D(t) \\ z_D(t) \end{bmatrix}. \quad (9)$$

The relative target position is

$$\boxed{\mathbf{p}_{rel}(t) = \mathbf{p}_T(t) - \mathbf{p}_D(t)} \quad (10)$$

and the relative distance is

$$d(t) = \|\mathbf{p}_{rel}(t)\|. \quad (11)$$

The controller does not directly use this 3-D distance. Instead, it estimates relative distance indirectly from the target’s apparent image size.

4 3. Camera Projection

Let the target position in the camera coordinate system be

$$\mathbf{p}_{TC} = \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}. \quad (12)$$

A perspective camera projects this 3-D point onto image coordinates:

$$\boxed{u = f_x \frac{X_C}{Z_C} + c_x} \quad (13)$$

and

$$\boxed{v = f_y \frac{Y_C}{Z_C} + c_y}. \quad (14)$$

Here (u, v) are pixel coordinates, f_x, f_y are focal lengths, and (c_x, c_y) is the principal point.

PyBullet performs the actual rendering in the implementation, but these equations explain mathematically why changes in UAV position and orientation change the target’s image position.

5 4. HSV Color Segmentation

The RGB camera image is converted to HSV:

$$I_{RGB}(u, v) \rightarrow I_{HSV}(u, v). \quad (15)$$

The default green-target threshold is

$$35 \leq H \leq 85, \quad (16)$$

$$80 \leq S \leq 255, \quad (17)$$

$$80 \leq V \leq 255. \quad (18)$$

The binary mask is therefore

$$M(u, v) = \begin{cases} 1, & 35 \leq H \leq 85, 80 \leq S \leq 255, 80 \leq V \leq 255, \\ 0, & \text{otherwise.} \end{cases} \quad (19)$$

Morphological erosion and dilation are then applied to reduce small noisy regions.

6 5. Target Detection

Contours are extracted from the binary mask. The largest valid contour is selected:

$$C^* = \operatorname{argmax}_{C_i} A(C_i) \quad (20)$$

subject to

$$A(C_i) \geq 15. \quad (21)$$

A minimum enclosing circle is fitted to the selected contour. The visual measurement returned by the tracker is

$$\mathbf{s} = \begin{bmatrix} c_x \\ c_y \\ r_{px} \end{bmatrix} \quad (22)$$

where c_x, c_y are the target centre coordinates in pixels and r_{px} is its apparent radius.

7 6. Desired Visual State

The camera resolution is

$$W = 320, \quad H = 240. \quad (23)$$

Hence the desired image centre is

$$c_x^* = \frac{W}{2} = 160, \quad (24)$$

$$c_y^* = \frac{H}{2} = 120. \quad (25)$$

The controller uses a desired target radius of

$$r_{px}^* = 45. \quad (26)$$

Therefore,

$$\mathbf{s}^* = \begin{bmatrix} 160 \\ 120 \\ 45 \end{bmatrix}. \quad (27)$$

The overall visual objective is

$$\mathbf{s} \rightarrow \mathbf{s}^* \quad (28)$$

or

$$c_x \rightarrow 160, \quad c_y \rightarrow 120, \quad r_{px} \rightarrow 45. \quad (29)$$

8 7. Visual Tracking Errors

The horizontal error is

$$e_x = c_x - 160. \quad (30)$$

The vertical error is

$$e_z = 120 - c_y. \quad (31)$$

The distance-related error is

$$e_d = 45 - r_{px}. \quad (32)$$

Thus, the visual error vector can be written as

$$\mathbf{e} = \begin{bmatrix} e_x \\ e_z \\ e_d \end{bmatrix}. \quad (33)$$

Successful tracking corresponds to

$$\mathbf{e} \rightarrow \mathbf{0}. \quad (34)$$

9 8. Apparent Target Size and Distance

The project uses the apparent radius of the target as an indirect distance measurement.

For a target of physical radius R_T , perspective projection gives approximately

$$\boxed{r_{px} \approx \frac{f R_T}{Z}} \quad (35)$$

where f is the camera focal length and Z is target depth from the camera. Therefore,

$$r_{px} \propto \frac{1}{Z}. \quad (36)$$

Hence,

$$r_{px} \downarrow \Rightarrow Z \uparrow \quad (37)$$

and

$$r_{px} \uparrow \Rightarrow Z \downarrow. \quad (38)$$

This explains why the controller can use target size to determine whether the UAV should move forward or backward, even though the code does not explicitly calculate Z .

10 9. PD Control Law

The class in the code is named `PIDController`, but the integral gain is set to zero:

$$K_I = 0. \quad (39)$$

Therefore, the implemented controller is mathematically a PD controller.

The discrete-time control law is

$$\boxed{u(k) = K_P e(k) + K_D \frac{e(k) - e(k-1)}{\Delta t}}. \quad (40)$$

The control frequency is

$$f_{ctrl} = 48 \text{ Hz}, \quad (41)$$

so

$$\boxed{\Delta t = \frac{1}{48} \approx 0.02083 \text{ s}}. \quad (42)$$

The proportional term responds to the current error, while the derivative term responds to the rate of change of the error and helps reduce oscillation.

11 10. Yaw Control

The horizontal error controls the UAV yaw rate.

The implemented gains are

$$K_{P,yaw} = 0.010, \quad K_{D,yaw} = 0.002. \quad (43)$$

Therefore,

$$\dot{\psi} = \text{sat}_{[-2,2]} \left[0.010e_x + 0.002 \frac{e_x(k) - e_x(k-1)}{\Delta t} \right]. \quad (44)$$

The objective is

$$c_x \rightarrow 160. \quad (45)$$

12 11. Altitude Control

The vertical error controls UAV vertical velocity.

The gains are

$$K_{P,z} = 0.008, \quad K_{D,z} = 0.002. \quad (46)$$

Thus,

$$v_z = \text{sat}_{[-0.8,0.8]} \left[0.008e_z + 0.002 \frac{e_z(k) - e_z(k-1)}{\Delta t} \right]. \quad (47)$$

The objective is

$$c_y \rightarrow 120. \quad (48)$$

13 12. Forward/Backward Control

The distance-related error controls forward and backward velocity.

The gains are

$$K_{P,d} = 0.05, \quad K_{D,d} = 0.01. \quad (49)$$

The command is

$$v_x = \text{sat}_{[-1.5,1.5]} \left[0.05e_d + 0.01 \frac{e_d(k) - e_d(k-1)}{\Delta t} \right]. \quad (50)$$

If $r_{px} < 45$, then $e_d > 0$ and the controller commands forward motion.

If $r_{px} > 45$, then $e_d < 0$ and the controller commands backward motion.

Thus,

$$r_{px} \rightarrow 45 \quad (51)$$

represents maintaining the desired apparent distance.

14 13. Body-to-World Velocity Transformation

The forward command v_x is defined relative to the UAV's current heading.

Let the UAV rotation matrix be $\mathbf{R}_D$. Its forward direction in world coordinates is

$$\mathbf{f}_D = \mathbf{R}_D \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}. \quad (52)$$

The commanded world-frame velocity is

$$\mathbf{v}_{world} = v_x \mathbf{f}_D + \begin{bmatrix} 0 \\ 0 \\ v_z \end{bmatrix}. \quad (53)$$

This transformation is essential because the UAV's forward direction changes when its yaw changes.

15 14. Desired Position and Yaw

The tracking velocity is converted into a short-term desired position using discrete integration:

$$\mathbf{p}_{cmd}(k+1) = \mathbf{p}_D(k) + \mathbf{v}_{world}(k)\Delta t. \quad (54)$$

The commanded yaw is updated by integrating the yaw rate:

$$\psi_{cmd}(k+1) = \psi_{cmd}(k) + \dot{\psi}(k)\Delta t. \quad (55)$$

The desired attitude supplied to the low-level controller is

$$\boldsymbol{\phi}_{cmd} = \begin{bmatrix} 0 \\ 0 \\ \psi_{cmd} \end{bmatrix}. \quad (56)$$

The low-level `DSLPIIDControl` controller then converts the desired position and orientation into the drone's motor commands.

16 15. Searching and Tracking

The system has two states:

$$\boxed{\text{SEARCHING} \quad \text{and} \quad \text{TRACKING.}} \quad (57)$$

When the target is detected,

$$\text{found} = 1, \quad (58)$$

the controller operates in `TRACKING` mode.

When the target is not detected,

$$\text{found} = 0, \quad (59)$$

the controller enters SEARCHING mode and commands

$$\boxed{v_x = 0, \quad v_y = 0, \quad v_z = 0, \quad \dot{\psi} = 0.3.} \quad (60)$$

This produces a slow yaw rotation to reacquire the target.

17 16. Closed-Loop System

The complete system can be represented mathematically by

$$\mathbf{s}(k) = h(\mathbf{x}_D(k), \mathbf{p}_T(k)), \quad (61)$$

where $h(\cdot)$ represents the camera and target-detection process.
The visual error is

$$\mathbf{e}(k) = \mathbf{s}(k) - \mathbf{s}^*. \quad (62)$$

The controller produces

$$\mathbf{u}(k) = \begin{bmatrix} v_x(k) \\ v_z(k) \\ \dot{\psi}(k) \end{bmatrix}. \quad (63)$$

The UAV state evolves according to

$$\mathbf{x}_D(k+1) = f(\mathbf{x}_D(k), \mathbf{u}(k)). \quad (64)$$

The new UAV state produces a new camera observation:

$$\mathbf{s}(k+1) = h(\mathbf{x}_D(k+1), \mathbf{p}_T(k+1)). \quad (65)$$

Therefore, the closed-loop operation is

$$\boxed{\mathbf{s}(k) \rightarrow \mathbf{e}(k) \rightarrow \mathbf{u}(k) \rightarrow \mathbf{x}_D(k+1) \rightarrow \mathbf{s}(k+1).} \quad (66)$$

This loop is repeated continuously throughout the simulation.

18 17. Overall Tracking Objective

A mathematical cost function that describes the tracking objective is

$$\boxed{J = w_x e_x^2 + w_z e_z^2 + w_d e_d^2} \quad (67)$$

where $w_x, w_z, w_d > 0$ are weighting factors.

The ideal tracking condition is

$$e_x = 0, \quad e_z = 0, \quad e_d = 0. \quad (68)$$

Equivalently,

$$\boxed{c_x = 160, \quad c_y = 120, \quad r_{px} = 45.} \quad (69)$$

The implementation does not explicitly optimize J ; this equation mathematically describes the desired behaviour of the controller.

19 18. Overall Mathematical Interpretation

The entire project can be summarized as follows:

$$\boxed{\text{Moving Target} \rightarrow \text{3-D Camera Observation} \rightarrow \text{HSV Segmentation} \rightarrow (c_x, c_y, r_{px}) \rightarrow (e_x, e_z, e_d) \rightarrow \text{PD}} \quad (70)$$

The fundamental tracking condition is

$$\boxed{\lim_{k \rightarrow \infty} \begin{bmatrix} c_x(k) - 160 \\ c_y(k) - 120 \\ r_{px}(k) - 45 \end{bmatrix} = \mathbf{0}.} \quad (71)$$

In practical terms, the UAV continuously adjusts its yaw, altitude, and forward/backward movement so that the moving target remains centred in the camera image and maintains the desired apparent size.

20 19. Key Mathematical Relationships

The most important relationships in the project are:

$$\boxed{c_x \rightarrow 160 \quad \Rightarrow \quad \text{horizontal alignment}} \quad (72)$$

$$\boxed{c_y \rightarrow 120 \quad \Rightarrow \quad \text{vertical alignment}} \quad (73)$$

$$\boxed{r_{px} \rightarrow 45 \quad \Rightarrow \quad \text{desired relative distance}} \quad (74)$$

and

$$\boxed{u = K_P e + K_D \frac{\Delta e}{\Delta t} \quad \Rightarrow \quad \text{corrective UAV motion.}} \quad (75)$$

Together, these equations explain the mathematical working of the vision-based UAV tracking system.